🦅 IRON — Falcon-Themed Crash Game

This document contains the complete, fully functional HTML5 source code for the **IRON Crash Game**. It features a dark-themed UI (Aviator style), dynamic Canvas animation with a trajectory curve, active multiplier updates, automatic cashout logic, and an endless game loop.

Component	Implementation Details
User Interface	HTML5 / CSS3 Flexbox (Mobile-first responsive frame)
Graphics Engine	HTML5 2D Canvas API (Grid background, curve filling, dynamic line rendering)
Game State Loop	Dynamic phase transitions (`WAITING` → `FLYING` → `CRASHED`) using `requestAnimationFrame`
Logic & Calculations	Dynamic exponential multiplier flight math, auto cashout checks, localized betting state

```
<!DOCTYPE html>
<html lang="hi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>IRON - बाज Crash Game</title>
    <style>
```

```
* { box-sizing: border-box; margin: 0; padding: 0; font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; }

body { background-color: #0b0e14; color: #fff; display: flex; flex-direction: column; align-items: center; min-height: 100vh; padding: 10px; }

header { width: 100%; max-width: 500px; display: flex; justify-content: space-between; align-items: center; background: #151922; padding: 12px 20px; border-radius: 12px; margin-bottom: 10px; border: 1px solid #222938; }

.logo { font-size: 20px; font-weight: bold; color: #f3ba2f; display: flex; align-items: center; gap: 8px; }

.wallet { background: #1f2633; padding: 6px 14px; border-radius: 20px; font-size: 14px; color: #4cd137; font-weight: bold; }



.game-container { width: 100%; max-width: 500px; background: #151922; border-radius: 16px; border: 1px solid #222938; overflow: hidden; display: flex; flex-direction: column; box-shadow: 0 8px 24px rgba(0,0,0,0.5); }



.canvas-box { position: relative; width: 100%; height: 260px; background: #0d1117; display: flex; justify-content: center; align-items: center; overflow: hidden; }

canvas { width: 100%; height: 100%; }

.multiplier-display { position: absolute; font-size: 42px; font-weight: bold; color: #fff; text-shadow: 0 2px 10px rgba(0,0,0,0.8); pointer-events: none; }

.status-text { position: absolute; top: 15px; font-size: 13px; color: #aaa; text-transform: uppercase; letter-spacing: 1px; font-weight: bold; }

.control-panel { padding: 15px; display: flex; flex-direction: column; gap: 12px; background: #151922; }

.bet-box { display: flex; justify-content: space-between; gap: 10px; background: #0d1117; padding: 10px; border-radius: 12px; border: 1px solid #222938; }

.bet-inputs { display: flex; flex-direction: column; width: 50%; }

.bet-inputs label { font-size: 11px; color: #aaa; margin-bottom: 4px; }
```

```
        .bet-inputs input { background: #151922; border: 1px solid #333; color: #fff; padding: 8px; border-radius: 8px; font-size: 16px; outline: none; text-align: center; }

        .action-btn { width: 100%; padding: 14px; font-size: 18px; font-weight: bold; border: none; border-radius: 12px; cursor: pointer; transition: 0.2s; text-transform: uppercase; }

        .btn-bet { background: #2ed573; color: #000; }

        .btn-bet:active { background: #26af5f; }

        .btn-cashout { background: #ff4757; color: #fff; display: none; }

        .btn-cashout:active { background: #e84118; }

        .history-bar { display: flex; gap: 6px; padding: 10px 15px; overflow-x: auto; background: #0d1117; border-top: 1px solid #222938; }

        .history-item { background: #1f2633; padding: 4px 10px; border-radius: 6px; font-size: 12px; font-weight: bold; color: #f3ba2f; white-space: nowrap; }
    </style>
</head>
<body>
    <header>
        <div class="logo">🦅 IRON</div>
        <div class="wallet" id="walletDisplay">₹1000.00</div>
    </header>
    <div class="game-container">
        <div class="canvas-box">
            <div class="status-text" id="statusText">अगला राउंड शुरू होने वाला है...</div>
```

```
        <div class="multiplier-display" id="multiplierDisplay">1.00x</div>
        <canvas id="gameCanvas" width="500" height="260"></canvas>
    </div>
    <div class="control-panel">
        <div class="bet-box">
            <div class="bet-inputs">
                <label>बेट अमाउंट (₹)</label>
                <input type="number" id="betAmount" value="100" min="10">
            </div>
            <div class="bet-inputs">
                <label>ऑटो कैश-आउट</label>
                <input type="number" id="autoCashout" placeholder="जैसे 2.00" step="0.1">
            </div>
        </div>
        <button class="action-btn btn-bet" id="actionBtn" onclick="handleBetAction()">बेट लगाएं</button>
    </div>
    <div class="history-bar" id="historyBar">
        <div class="history-item">2.45x</div>
        <div class="history-item">1.20x</div>
        <div class="history-item">5.10x</div>
    </div>
```

```
</div>
<script>
    const canvas = document.getElementById("gameCanvas");
    const ctx = canvas.getContext("2d");
    const multiplierDisplay = document.getElementById("multiplierDisplay");
    const statusText = document.getElementById("statusText");
    const actionBtn = document.getElementById("actionBtn");
    const walletDisplay = document.getElementById("walletDisplay");
    const betAmountInput = document.getElementById("betAmount");
    const autoCashoutInput = document.getElementById("autoCashout");
    const historyBar = document.getElementById("historyBar");
    let walletBalance = 1000.0;
    let currentBet = 0;
    let gameState = "WAITING";
    let multiplier = 1.00;
    let crashPoint = 1.00;
    let isPlaying = false;
    let hasCashedOut = false;
    let flightAnimId = null;
    let pathPoints = [];
    function drawScene(currentMult, state, crash = false) {
        ctx.clearRect(0, 0, canvas.width, canvas.height);
```

```
ctx.strokeStyle = "#1b2230";
ctx.lineWidth = 1;
for (let i = 0; i < canvas.width; i += 50) {
    ctx.beginPath(); ctx.moveTo(i, 0); ctx.lineTo(i, canvas.height); ctx.stroke();
}
for (let j = 0; j < canvas.height; j += 40) {
    ctx.beginPath(); ctx.moveTo(0, j); ctx.lineTo(canvas.width, j); ctx.stroke();
}
if (state === "FLYING" || state === "CRASHED") {
    let progressX = Math.min(canvas.width - 40, 30 + (currentMult - 1.0) * 70);
    let progressY = Math.max(30, canvas.height - 30 - Math.pow(currentMult - 1.0, 1.2) * 50);
    pathPoints.push({x: progressX, y: progressY});
    if (pathPoints.length > 1) {
        ctx.beginPath();
        ctx.moveTo(30, canvas.height - 30);
        for (let p of pathPoints) {
            ctx.lineTo(p.x, p.y);
        }
        ctx.lineTo(progressX, canvas.height - 30);
        ctx.closePath();
        ctx.fillStyle = "rgba(255, 38, 38, 0.18)";
        ctx.fill();
```

```
            ctx.beginPath();
            ctx.moveTo(30, canvas.height - 30);
            for (let p of pathPoints) {
                ctx.lineTo(p.x, p.y);
            }
            ctx.strokeStyle = "#ff1e1e";
            ctx.lineWidth = 4.5;
            ctx.stroke();
        }
        if (!crash) {
            ctx.font = "26px Arial";
            ctx.fillText("🦅", progressX - 12, progressY + 10);
        } else {
            ctx.font = "30px Arial";
            ctx.fillText("💥", progressX - 15, progressY + 10);
        }
    }
}
function startWaitingPhase() {
    gameState = "WAITING";
    statusText.innerText = "अगला राउंड शुरू होने वाला है...";
    multiplierDisplay.innerText = "1.00x";
```

```
        multiplierDisplay.style.color = "#fff";
        pathPoints = [];
        drawScene(1.00, "WAITING");
        let countdown = 3;
        let timer = setInterval(() => {
            countdown--;
            if (countdown > 0) {
                statusText.innerText = `अगला राउंड ${countdown} सेकंड में...`;
            } else {
                clearInterval(timer);
                startFlyingPhase();
            }
        }, 1000);
    }
    function startFlyingPhase() {
        gameState = "FLYING";
        statusText.innerText = "बाज उड़ रहा है...";
        multiplier = 1.00;
        pathPoints = [];

        crashPoint = parseFloat((Math.random() < 0.12 ? (Math.random() * 0.5 + 1.01) :
(Math.random() * 8 + 1.2)).toFixed(2));
        if (isPlaying && !hasCashedOut) {
```

```
        actionBtn.innerText = "कैश आउट";
        actionBtn.className = "action-btn btn-cashout";
    }
    let startTime = Date.now();
    function updateFlight() {
        if (gameState !== "FLYING") return;
        let elapsed = (Date.now() - startTime) / 1000;
        multiplier = parseFloat((1.00 + Math.pow(elapsed, 1.4) * 0.35).toFixed(2));
        multiplierDisplay.innerText = multiplier.toFixed(2) + "x";
        drawScene(multiplier, "FLYING");
        let autoVal = parseFloat(autoCashoutInput.value);
        if (isPlaying && !hasCashedOut && autoVal && multiplier >= autoVal) {
            cashOut();
        }
        if (multiplier >= crashPoint) {
            triggerCrash();
        } else {
            flightAnimId = requestAnimationFrame(updateFlight);
        }
    }
    flightAnimId = requestAnimationFrame(updateFlight);
}
```

```
function triggerCrash() {
    gameState = "CRASHED";
    cancelAnimationFrame(flightAnimId);
    multiplierDisplay.innerText = `${crashPoint.toFixed(2)}x`;
    multiplierDisplay.style.color = "#ff1e1e";
    statusText.innerText = "बाज क्रैश हो गया!";
    drawScene(crashPoint, "CRASHED", true);
    if (isPlaying && !hasCashedOut) {
        isPlaying = false;
        actionBtn.innerText = "बेट लगाएं";
        actionBtn.className = "action-btn btn-bet";
    }
    addHistory(crashPoint);
    setTimeout(startWaitingPhase, 4000);
}
function handleBetAction() {
    let amount = parseFloat(betAmountInput.value);

    if (gameState === "WAITING") {
        if (isNaN(amount) || amount <= 0 || amount > walletBalance) {
            alert("अमान्य बेट राशि!");
            return;
```

```
        }
        currentBet = amount;
        walletBalance -= currentBet;
        walletDisplay.innerText = `₹${walletBalance.toFixed(2)}`;
        isPlaying = true;
        hasCashedOut = false;

        actionBtn.innerText = "बेट स्वीकार हो गई";
        actionBtn.style.background = "#ffa502";
        actionBtn.disabled = true;
        setTimeout(() => { actionBtn.disabled = false; }, 500);
    }
    else if (gameState === "FLYING" && isPlaying && !hasCashedOut) {
        cashOut();
    }
}
function cashOut() {
    hasCashedOut = true;
    let winAmount = currentBet * multiplier;
    walletBalance += winAmount;
    walletDisplay.innerText = `₹${walletBalance.toFixed(2)}`;
    actionBtn.innerText = `जीत गए! ₹${winAmount.toFixed(2)}`;
```

```
            actionBtn.className = "action-btn btn-bet";
            actionBtn.style.background = "#2ed573";
            isPlaying = false;
        }
        function addHistory(mult) {
            let item = document.createElement("div");
            item.className = "history-item";
            item.innerText = `${mult.toFixed(2)}x`;
            historyBar.prepend(item);
            if (historyBar.children.length > 8) {
                historyBar.lastChild.remove();
            }
        }
        startWaitingPhase();
    </script>
</body>
</html>
```